Introduction of Automata Theory

What is TOC???

In theoretical computer science, the **theory of computation** is the branch that deals with whether and how efficiently problems can be solved on a model of computation, using an algorithm.

In theoretical computer science, automata theory is the study of abstract machines (or more appropriately, abstract 'mathematical' machines or systems) and the computational problems that can be solved using these machines. These abstract machines are called automata.

Automata Theory

- In theory of Computation, Computation means processing.
- Processing input towards the ouput.
- In every machine there will be a low level calculations which we call it as Abstract machine or a Model
- Abstract machine takes input process it and gives output which cannot be seen.
- Eg. Electric Switch Model
- This abstract machine or model is called Automata
- If this machine implements the model in a finite number of states then it is called Finite Automata.

The field is divided into three major branches:

- 1. Automata theory,
  2. Computability theory and
  3. Computational complexity theory.

Automata theory

- It deals with the definition and properties of various mathematical model of computation.
- It deals with developing a model which takes input, process it to give an ouput.

Eg.

- Finite Automata
- Context free Language
- Turing machines

Computability theory

- It deals with what can and what cannot be computed by the model
- It deals or checks the ability of a computing

Computational Complexity Theory

- It groups the computability problem based on their hardness.
- Every problem will have no. of solutions and selecting the optimal solution among the solutions.
- Eg. NP model

Main purpose of Theory of computation?

- To develop formal mathematical model of computation that reflect real world computer.
- This theory is essential in the field of computer science as it forms the basis for designing and analyzing algorithms, programming languages, and compilers.

Automata theory

- This automaton consists of
- • **states** (represented in the figure by circles),
- • and **transitions** (represented by arrows).
- As the automaton sees a symbol of input, it makes a *transition* (or *jump*) to another state,
- according to its ***transition function*** (which takes the current state and the recent symbol as its
- inputs).

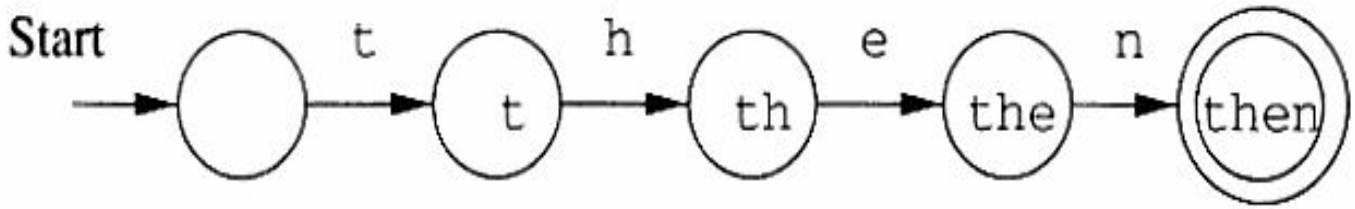

Figure 1.2: A finite automaton modeling recognition of **then**

Applications

- Design of a compiler
- Pattern Matching
- Spell Check

The Central Concepts of Automata Theory

- Symbols
- Alphabets
- Strings
- Language

Symbols

- A symbol (often also called a **character**) is the smallest building block, which can be any alphabet, letter, or picture.

a, b, c, 0, 1, ...

Alphabets (Σ(sigma)):

- Alphabets are a set of symbols, which are always ***finite***.
- It is also called binary alphabet.

Examples:

$\Sigma = \{0, 1\}$ is an alphabet of binary digits

$\Sigma = \{0, 1, \ldots\ldots, 9\}$ is an alphabet of decimal digits

$\Sigma = \{a, b, c\}$

$\Sigma = \{A, B, C, \ldots\ldots\ldots, Z\}$

Strings

- A string is a ***finite*** sequence of symbols from some alphabet. A string is generally denoted as ***w*** and the length of a string is denoted as **|*w*| *(Mod W).***
- Empty string is the string with zero occurrence of symbols,
- represented as ε(epsilon).
- **Number of Strings (of length 2) that can be generated over the alphabet {a, b}:**
- a a, a b, b a, b b
- Length of String |w| = 2
- Number of Strings = 4

String:

Powers of an Alphabet

If Σ is an alphabet, we can express the set of all strings of a certain length from that alphabet by using an exponential notation. We define Σ^k to be the set of strings of length k, each of whose symbols is in Σ.

Example 1.24: Note that $\Sigma^0 = \{\epsilon\}$, regardless of what alphabet Σ is. That is, ϵ is the only string whose length is 0.

If $\Sigma = \{0, 1\}$, then $\Sigma^1 = \{0, 1\}$, $\Sigma^2 = \{00, 01, 10, 11\}$,

$$\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$$

String:

- **Closure Representation in TOC:**

- **L^+**: It is a ***Positive Closure*** that represents a set of all strings except Null or ε-strings.
- **L^***: It is "**Kleene Closure**", that represents the occurrence of certain alphabets for given language alphabets from zero to the infinite number of times. In which ε-string is also included.
- From the above two statements, it can be concluded that:
- $\mathbf{L^* = \varepsilon L^+}$

String:

The set of all strings over an alphabet Σ is conventionally denoted Σ^*. For instance, $\{0,1\}^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, \ldots\}$. Put another way,

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \cdots$$

Sometimes, we wish to exclude the empty string from the set of strings. The set of nonempty strings from alphabet Σ is denoted Σ^+. Thus, two appropriate equivalences are:

- $\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \cdots$.
- $\Sigma^* = \Sigma^+ \cup \{\epsilon\}$.

Concatenation of Strings

Let x and y be strings. Then xy denotes the *concatenation* of x and y, that is, the string formed by making a copy of x and following it by a copy of y. More precisely, if x is the string composed of i symbols $x = a_1 a_2 \cdots a_i$ and y is the string composed of j symbols $y = b_1 b_2 \cdots b_j$, then xy is the string of length $i + j$: $xy = a_1 a_2 \cdots a_i b_1 b_2 \cdots b_j$.

Example 1.25: Let $x = 01101$ and $y = 110$. Then $xy = 01101110$ and $yx = 11001101$. For any string w, the equations $\epsilon w = w\epsilon = w$ hold. That is, ϵ is the *identity for concatenation*, since when concatenated with any string it yields the other string as a result (analogously to the way 0, the identity for addition, can be added to any number x and yields x as a result). $\square$

Language

- A language is a *set of strings*, chosen from some Σ* or we can say- ‘A language is a subset of Σ* ‘. A language that can be formed over ‘ Σ ‘ can be **Finite** or **Infinite**.
- Example of Finite Language:
- L1 = { set of string of 2 }
- L1 = { xy, yx, xx, yy }
- Example of Infinite Language:
- L1 = { set of all strings starts with 'b' }
- L1 = { babb, baa, ba, bbb, baab, ... }